

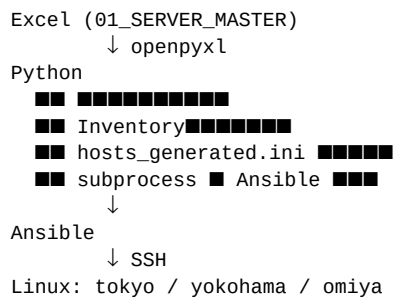
Linux Server Operations Lab

Phase 4 復習資料：Excel → Python → Ansible Inventory 自動生成

Phaseの目的	Excelのサーバ台帳をPythonで読み取り、Ansible Inventoryを自動生成し、そのInventoryでLinux 3台へ接続
今回の到達点	Excel → Python → hosts_generated.ini → Ansible → tokyo / yokohama / omiya が1回のPython実行でつながる
重要キーワード	Inventory / ansible_host / ansible_user / f文字列 / open() / write() / subprocess / returncode / diff

今回の全体像

Excelは「管理台帳」、Pythonは「変換・制御」、Ansibleは「複数Linuxサーバへ実行」、Linuxは「実際に操作される対象」という役割分担を確認した。



1. 何を作ったか・なぜ必要か

Phase 4では、Excelに登録した Hostname と IP_Address をもとに、Ansibleが接続先を認識するための Inventory をPythonで自動生成した。最初に手動版 hosts.ini を作り、正しい完成形を確認してから、自動生成版 hosts_generated.ini を作った。

なぜ最初に手動版を作ったのか

いきなりPythonで自動生成すると、失敗したときに「Inventoryの書式が悪いのか」「Pythonの生成処理が悪いのか」を切り分けにくい。先に手動版でAnsible疎通を成功させることで、「このInventory形式なら正しい」という基準を作った。

Inventory 1行の意味

```
[servers]
tokyo ansible_host=192.168.56.11 ansible_user=tokyo
```

tokyo はAnsible上のホスト名、ansible_host は実際の接続先IP、ansible_user はSSH接続ユーザーを表す。グループ名 [servers] を指定することで、ansible servers ... のように3台をまとめて対象にできる。

実務での用途

サーバ台数やIPが変わっても、Excelを更新してPythonを再実行すればInventoryを作り直せる設計の土台になる。Inventoryへの二重入力を減らし、管理台帳と自動化設定の不一致を防ぎやすくなる。

Excel関数について

Phase 4ではExcel関数は使用していない。ExcelはPhase 2で作成した tblServer のデータソースとして利用した。

2. 実行した主なコマンド

目的	コマンド	意味
Inventoryディレクトリ作成	<code>mkdir -p ~/linux-server-operations-lab/inventory</code>	Inventoryを役割別ディレクトリに分離する。
手動Inventory編集	<code>nano ~/linux-server-operations-lab/inventory/hosts.ini</code>	Python自動生成前に正しい完成形を作る。
手動Inventory疎通	<code>ansible servers -i ~/linux-server-operations-lab/inventory/hosts.ini --ping</code>	Inventory形式とSSH接続情報が正しいことを確認する。
生成Inventory確認	<code>cat ~/linux-server-operations-lab/inventory/hosts_generated.ini</code>	Pythonが書いたファイル内容を人間が確認する。
生成Inventory疎通	<code>ansible servers -i ~/linux-server-operations-lab/inventory/hosts_generated.ini --ping</code>	自動生成版がAnsibleで実行可能か確認する。
差分確認	<code>diff ~/linux-server-operations-lab/inventory/hosts.ini ~/linux-server-operations-lab/inventory/hosts_generated.ini</code>	手動版と自動生成版が完全一致するか確認する。

結果

手動版・自動生成版の両方で `tokyo / yokohama / omiya` が `SUCCESS`、`ping: pong` となった。diffは何も表示されず、2ファイルが完全一致した。

3. Pythonコードと1行ごとの意味

```
import openpyxl
import subprocess

excel_path = "/home/controller/Linux_Server_Operations_Lab.xlsx"
inventory_path = "/home/controller/linux-server-operations-lab/inventory/hosts_generated.ini"

wb = openpyxl.load_workbook(excel_path)
ws = wb["01_SERVER_MASTER"]
headers = [ws.cell(row=1, column=c).value for c in range(1, 7)]
servers = [
    dict(zip(headers, [ws.cell(row=r, column=c).value for c in range(1, 7)]))
    for r in range(2, 5)
]

with open(inventory_path, "w") as f:
    f.write("[servers]\n")
    for server in servers:
        f.write(
            f'{server["Hostname"]} '
            f'ansible_host={server["IP_Address"]} '
            f'ansible_user={server["Hostname"]}\n'
        )

result = subprocess.run(
    ["ansible", "servers", "-i", inventory_path, "-m", "ping"],
    capture_output=True,
    text=True
)

print(result.stdout)
print("Ansible returncode:", result.returncode)
```

コード要素	意味
import openpyxl	Excel .xlsx を読み取るライブラリを読み込む。
import subprocess	Pythonから外部コマンド（今回はansible）を実行する。
load_workbook	Excelブックを開く。
ws = wb[...]	01_SERVER_MASTER シートを選ぶ。
headers	1行目の列名を取得する。
dict(zip(...))	列名と各行の値を組み合わせ、1台分を辞書型にする。
with open(..., "w")	Inventoryファイルを書き込みモードで開く。既存内容は上書きされる。
f.write	文字列をファイルへ書き込む。
f文字列	Excel由来のHostname/IPをInventory書式へ埋め込む。
subprocess.run	生成したInventoryを使ってAnsible pingを実行する。
capture_output=True	Ansibleの標準出力・標準エラーをPythonで受け取れるようにする。
text=True	出力を文字列として扱う。
returncode	0ならコマンド正常終了、0以外なら異常終了の判断材料になる。

4. トラブル・原因候補・切り分け

事象	原因候補	切り分け	修正 / 再確認
AnsibleでUNREACHABLE	IP誤り、SSHユーザー誤り、SSH停止	手動問題 inventoryで ping → ssh → ansible pingの順に確認	pingの順に確認し、再生成後にansible -m ping
自動生成Inventoryが壊れる	f文字列、改行、列名の誤り	cat hosts_generated.ini で内容を目視	Pythonの生成部を修正し、再生成してcat
手動版と自動版が違う	Excel値または生成ロジックの差	diff hosts.ini hosts_generated.ini	差分行を特定してExcel/コードを修正
PythonからAnsibleが失敗	Inventoryは正しいがsubprocess引数が調ひ	InventoryでAnsible単体実行	subprocess.run の引数を見直し returncode を再確認
Excelの値が不正	Hostname/IPの空欄・表記揺れ	Phase 3のバリデーションを先に通	異常データをAnsibleへ渡さない

今回実際に確認した切り分けの順番

1. ■■■ hosts.ini ■■■
2. ■■■Inventory■Ansible ping■■■■■
3. Python■Inventory■■■■■■■■■
4. Python■ hosts_generated.ini ■■■
5. cat ■■■■■
6. ■■■■■■Ansible ping
7. diff ■■■■■■■■■■
8. Python subprocess ■■■Ansible■■■
9. ■■■■Excel■■■→■■■→Ansible■■■1■■■■■

この順番の利点

問題が起きた場所を段階ごとに分離できる。いきなり全部を自動化せず、各層で正しさを確認してから次へ進むことで、「何が悪いかな」を短時間で特定しやすい。

5. 結果・学んだこと・実務へのつながり

最終結果

```
Inventory generated: /home/controller/linux-server-operations-lab/inventory/hosts_generated.ini
```

```
tokyo      | SUCCESS => ping: pong
yokohama   | SUCCESS => ping: pong
omiya      | SUCCESS => ping: pong
```

```
Ansible returncode: 0
```

学んだこと

- ・ Excelの台帳データは、そのままではAnsibleに使えない。Pythonが「変換レイヤー」として橋渡しする。
- ・ Inventoryは単なる接続先一覧ではなく、Ansibleが「誰に・どのIPへ・どのユーザーで」接続するかを定義する重要ファイル。
- ・ 自動化では、正しい完成形を先に手動で確認してから自動生成すると切り分けが容易になる。
- ・ Pythonのf文字列・ファイル書き込み・subprocessを組み合わせると、Excelの変更をAnsible実行へつなげられる。
- ・ diffのような単純なLinuxコマンドも、自動生成結果の品質確認に非常に有効。
- ・ 最終的に「Excel → Python → Ansible → Linux」という本プロジェクトの中核フローが初めて一続きになった。

実務での使い方

運用部門がExcelでサーバー一覧を管理している場合、PythonでInventoryを自動生成すれば、台帳更新後の転記作業を減らせる。サーバ追加・IP変更・環境区分の変更を自動化基盤へ反映する入口として使える。ただし本番では、入力検証、バックアップ、Git管理、権限制御、秘密情報の分離などを追加する必要がある。

Phase 5への接続

次のPhase 5では、この自動生成Inventoryを使って uptime / CPU / memory / disk / OS / IP / nginxなどの日次点検をAnsibleで実行する。Phase 4で「対象サーバを決める仕組み」ができたため、次は「対象へ何を点検させるか」を作る段階に進む。

6. note記事候補

種類	タイトル案	記事で伝えるポイント
候補1	Excelのサーバ台帳からAnsible InventoryをPythonで自動生成してみたい	Excelをみたすることで、Ansibleを知らない人にも「台帳から自動化
候補2	AnsibleのInventoryを手書きするのをやめて、Excelから自動生成After型	手入力の手入力の転記ミスや二重管理を減らす切り口。
候補3	Excel → Python → Ansible → Linux が初めてつながった日	学習記録型。Phase 1〜3の積み上げがPhase 4でつながるストーリーにて
候補4	自動化する前に「手動で正解を作る」理由	今回の手動hosts.ini→自動生成→diffという切り分け手法を初心者向け

復習チェック

- ☐ Inventoryの `ansible_host` と `ansible_user` を説明できるか。
- ☐ なぜ手動Inventoryを先に作ったのか説明できるか。
- ☐ Excelの1行がPythonの辞書へ変換され、f文字列でInventory行になる流れを説明できるか。
- ☐ diff が何も表示しなかった意味を説明できるか。
- ☐ Pythonの `subprocess` と `returncode` の役割を説明できるか。

Phase 4 完了 : Excel → Python → Inventory → Ansible → Linux の接続を確認済み。